



UNIVERSITAS MULTI DATA PALEMBANG

Fakultas Ilmu Komputer dan Rekayasa

Program Studi Sistem Informasi

DOKUMENTASI BUKTI PEKERJAAN

Kuis Refactor: Arsitektur Backend Terstruktur

Route, Controller, Model, dan Middleware pada RESTful API Tugas 1

Pengembangan Aplikasi Web II | SUB-CPMK-6 | Kelas SI5B

Nama	Yonathan Rinf
NPM	2428240101
Kelas	SI5B
Topik Tugas 1	Coworking Space - Reservasi Ruang Rapat (Topik 28)
Entitas utama	meeting-reservations (Reservasi Ruang Rapat)
Tautan repositori	https://github.com/Nathan-rar/si5b_kuis1_yonathan_rinf
Dosen pengampu	Nur Rachmat, M.Kom.
Tanggal pengumpulan	8 Oktober 2026

Petunjuk Pengisian

Dokumen ini adalah bukti bahwa Anda sendiri yang menyelesaikan *refactor* Tugas 1 ke arsitektur backend terstruktur. Dosen menilai dari dua hal: tangkapan layar (*screenshot*) yang membuktikan program berjalan, dan penjelasan Anda yang membuktikan bahwa Anda memahami kode tersebut.

A. Cara mengisi

1. Ganti seluruh tulisan dalam tanda kurung siku, misalnya *meeting-reservations* dan *Reservasi Ruang Rapat*, dengan topik Tugas 1 Anda. Contoh: untuk topik Buku, *meeting-reservations* menjadi buku.
2. Setiap langkah memiliki tiga bagian: *screenshot*, keterangan gambar, dan kotak **Penjelasan**. Ketiganya wajib diisi.
3. Tulis penjelasan dengan kalimat Anda sendiri. Penjelasan yang hanya menyalin nama berkas atau tidak menyebut kode topik Anda dianggap belum mengerti.
4. Pada pengujian, gunakan data milik topik Anda dan masukkan identitas Anda (nama atau NIM) pada salah satu field teks pada permintaan POST. Ini menjadi tanda bahwa gambar berasal dari pekerjaan Anda.
5. Simpan hasil akhir sebagai PDF dengan nama `PAW2_Kuis1_SI5B_[NIM]_[Nama].pdf`, lalu kumpulkan melalui LMS sesuai batas waktu yang ditentukan dosen.

B. Syarat tangkapan layar

Jenis	Yang harus terlihat
Terminal	Perintah yang diketik, seluruh keluarannya, dan nama folder project pada baris perintah.
Kode	Tidak perlu ditangkap layar. Kode dinilai dari tautan repositori GitHub.
Pengujian <i>endpoint</i>	Metode (GET, POST, PUT, DELETE), alamat lengkap, header yang dikirim, body (bila ada), kode status, dan isi respons. Boleh memakai Postman, Thunder Client, REST Client, atau <code>curl</code> di terminal.
Larangan	Gambar dipotong sehingga status atau isi respons tidak terlihat, gambar buram, gambar hasil unduhan dari teman atau internet, dan gambar yang diedit.

Cara mengambil screenshot: Windows tekan `Win + Shift + S`; macOS tekan `Cmd + Shift + 4`; Linux gunakan `PrtSc` atau aplikasi tangkapan layar bawaan. Tempel gambar ke kotak pada dokumen ini, lalu atur lebarnya agar seluruh tulisan masih terbaca.

C. Susunan dokumen

Bab	Isi	Bukti utama
Bab 1	Instalasi	1 screenshot terminal
Bab 2	Struktur folder dan ringkasan kode	1 screenshot struktur folder, tabel penjelasan

Bab	Isi	Bukti utama
Bab 3	Menjalankan server	1 screenshot terminal
Bab 4	Pengujian endpoint (5 berhasil, 4 gagal)	9 screenshot
Bab 5	Rekapitulasi hasil uji	Tabel rekap
Bab 6	Repositori kode	Tautan GitHub
Bab 7	Refleksi	Dua jawaban tertulis

Seluruh dokumen hanya memerlukan **12 tangkapan layar**. Ambil tangkapan layar sambil mengerjakan, bukan di akhir, dan sisakan sekitar 25 menit dari waktu kuis untuk melengkapi penjelasan.

Identitas Topik dan Rancangan Endpoint

Bagian ini menjembatani topik Tugas 1 Anda dengan contoh yang dipakai pada soal. Isilah sebelum mengerjakan bab berikutnya. Seluruh tanda [...] pada bab selanjutnya mengacu ke tabel ini.

Data topik

Topik Tugas 1	Coworking Space - Reservasi Ruang Rapat (Topik 28)
Entitas utama (tunggal)	Reservasi Ruang Rapat
Alamat dasar	meeting-reservations
Field data	id, namaPemesan, ruang, tanggal, jamMulai, durasiJam
Field wajib saat POST	namaPemesan, ruang, tanggal, jamMulai, durasiJam
Query string untuk filter	?tanggal=YYYY-MM-DD
Data awal	3 data awal: PT Maju Digital, CV Kreatif Nusantara, Tim Sistem Informasi
Modul lain (jika ada)	Tidak ada

Daftar endpoint

Tuliskan seluruh *endpoint* pada API Anda. Kolom "API key" diisi Ya bila rute dilindungi cekApiKey.

No	Metode	Alamat	API key	Status sukses	Fungsi
1	GET	Tidak	Tidak	200	Ambil semua data (dengan filter bila ada)
2	GET	Tidak	Tidak	200	Ambil satu data
3	POST	Ya	Ya	201	Tambah data
4	PUT	Ya	Ya	200	Ubah data
5	DELETE	Ya	Ya	204	Hapus data

Jika topik Anda memiliki *endpoint* tambahan atau modul lain (misalnya relasi antartabel), tambahkan barisnya di sini dan uji pada bagian "Skenario Tambahan" di Bab 4.

Bab 1 Instalasi

Bab ini membuktikan bahwa lingkungan kerja siap dan *project* dibuat dari awal. Cukup **satu** tangkapan layar terminal yang memuat seluruh perintah instalasi.

1.1 Instalasi project

Harus terlihat pada screenshot: perintah `node -v`, `npm -v`, `npm init -y`, dan `npm install express cors dotenv` beserta keluarannya sampai selesai.

```

v24.21.0
11.19.0
Wrote to /Users/nathan/Projects/si5b_kuis1_yonathan_rinfi/package.json:

{
  "name": "si5b_kuis1_yonathan_rinfi",
  "version": "1.0.0",
  "description": "Kuis 1 Pengembangan Aplikasi Web II - Refactor RESTful API Reservasi Ruang Rapat ke arsitektur Route, Controller, Model, dan Middleware",
  "main": "app.js",
  "scripts": {
    "start": "node app.js",
    "dev": "node --watch app.js"
  },
  "keywords": [
    "express",
    "restful-api",
    "coworking-space",
    "reservasi-ruang-rapat"
  ],
  "author": "Yonathan Rinfi",
  "license": "MIT",
  "dependencies": {
    "cors": "^2.8.5",
    "dotenv": "^18.0.6",
    "express": "^0.21.1"
  },
  "devDependencies": {},
  "repository": {
    "type": "git",
    "url": "git+https://github.com/Nathan-rar/si5b_kuis1_yonathan_rinfi.git"
  },
  "type": "commonjs",
  "bugs": {
    "url": "https://github.com/Nathan-rar/si5b_kuis1_yonathan_rinfi/issues"
  },
  "homepage": "https://github.com/Nathan-rar/si5b_kuis1_yonathan_rinfi#readme"
}

up to date, audited 72 packages in 484ms

30 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
nathan@MacBook-Air-Nathan: si5b_kuis1_yonathan_rinfi %

```

Gambar 1. Versi Node.js dan instalasi paket

Gambar 1. Versi Node.js dan instalasi paket

Penjelasan:

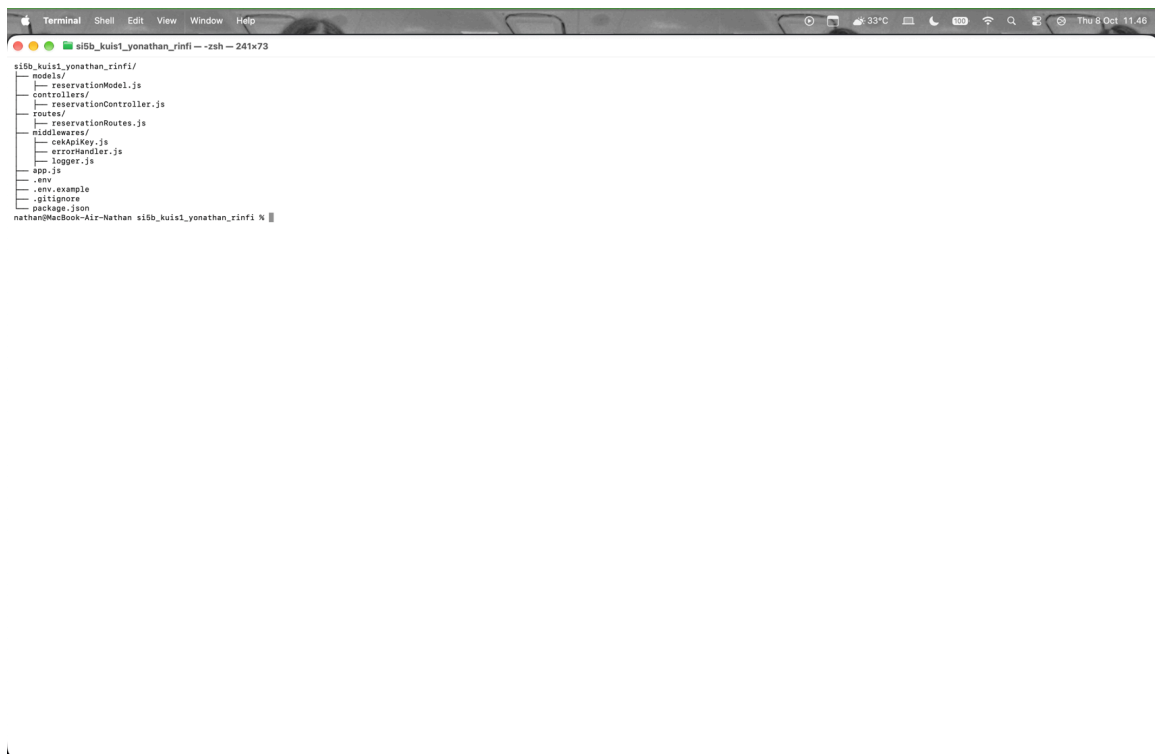
Pada proyek ini saya memakai tiga paket. *express* menjadi kerangka kerja pembuat server dan route, sehingga saya dapat menulis `app.get`, `app.post`, dan seterusnya. *cors* saya pasang lewat `app.use(cors())` supaya API dapat diakses dari domain atau port lain, misalnya ketika diuji dari Postman. *dotenv* saya pakai lewat `require("dotenv").config()` agar nilai `PORT` dan `API_KEY` dibaca dari berkas `.env`, bukan ditulis langsung di dalam kode. Versi Node.js perlu memenuhi syarat Express 5 karena Express 5 hanya mendukung Node.js 18 ke atas; di komputer saya Node.js v24.21.0, jadi sudah memenuhi syarat tersebut.

Bab 2 Struktur Folder dan Ringkasan Kode

Kode tidak perlu ditangkap layar. Dosen membaca kode langsung dari repositori (lihat Bab 6). Yang diminta di sini adalah **satu** tangkapan layar struktur folder dan penjelasan singkat tiap berkas dengan kalimat Anda sendiri.

2.1 Struktur folder

Harus terlihat pada screenshot: folder routes, controllers, models, dan middlewares beserta isinya. Gunakan panel *Explorer* editor atau perintah tree.



```

si5b_kuis1_yonathan_rinfi/
├── models/
│   └── reservationModel.js
├── controllers/
│   └── reservationController.js
├── routes/
│   └── reservationRoutes.js
├── middlewares/
│   ├── cekApiKey.js
│   ├── errorHandler.js
│   └── logger.js
├── app.js
├── .env
├── .env.example
├── .gitignore
└── package.json
nathan@MacBook-Air-Nathan si5b_kuis1_yonathan_rinfi %

```

Gambar 2. Struktur folder project

Gambar 2. Struktur folder project

Penjelasan:

Folder models menyimpan data beserta fungsi pengolahannya, folder controllers menangani request dan menyusun response, folder routes memetakan alamat ke fungsi controller, dan folder middlewares memuat kode yang berjalan sebelum atau sesudah request ditangani. Berkas app.js menjadi titik masuk yang merakit seluruh bagian tersebut.

2.2 Ringkasan tugas setiap berkas

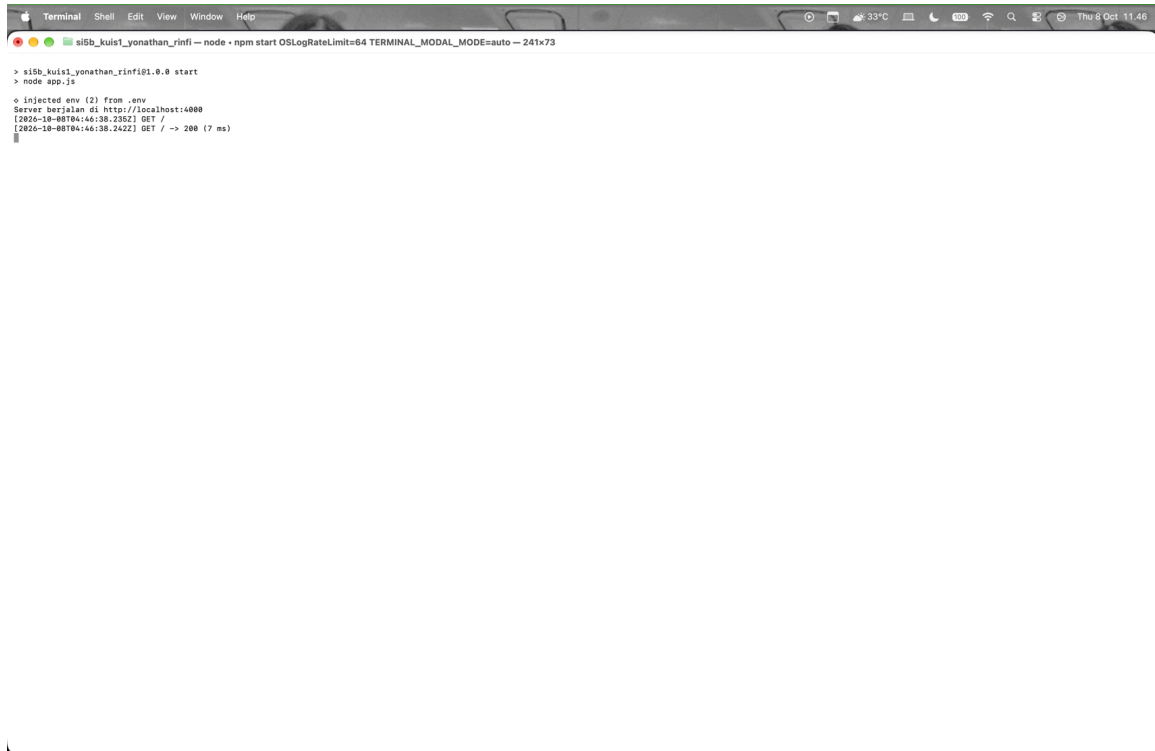
Tulis satu atau dua kalimat per berkas. Sebut nama fungsi atau variabel dari kode Anda.

Berkas	Penjelasan Anda
models/reservationModel.js	Menyimpan array meetingReservations dan fungsi findAll, findByld, create, update, serta remove. Berkas ini tidak memakai req maupun res.
controllers/reservationController.js	Memuat getAllReservations, getReservationByld, createReservation, updateReservation, deleteReservation, dan validateReservation untuk memeriksa field wajib.
routes/reservationRoutes.js	Memakai express.Router() untuk memetakan GET, POST, PUT, dan DELETE ke fungsi controller. Middleware cekApiKey dipasang pada rute POST, PUT, dan DELETE.
middlewares/logger.js	Fungsi logger mencatat metode, alamat, kode status, dan durasi setiap request melalui event res.on("finish").
middlewares/cekApiKey.js	Fungsi cekApiKey membandingkan header x-api-key dengan process.env.API_KEY. Bila tidak cocok, dibalas 401 dan next() tidak dipanggil.
middlewares/errorHandler.js	Berisi notFound untuk 404 rute yang tidak ada dan errorHandler empat parameter yang mengubah error menjadi JSON, termasuk SyntaxError dari express.json().
app.js	Merakit cors, logger, express.json(), route utama, lalu notFound dan errorHandler. Nilai PORT dibaca dari process.env.PORT.

Bab 3 Menjalankan Server

3.1 Menjalankan server

Harus terlihat pada screenshot: perintah `npm start` atau `npm run dev` dan pesan bahwa server berjalan beserta nomor port. Biarkan terminal ini tetap terbuka selama pengujian.



```
> si5b_kuis1_yonathan_rinfi@10.0.0 start
> node app.js

o injected env (2) from .env
Server berjalan di: http://localhost:4000
[2026-10-08T04:46:38.235Z] GET /
[2026-10-08T04:46:38.242Z] GET / -> 200 (7 ms)
```

Gambar 3. Server berhasil dijalankan

Gambar 3. Server berhasil dijalankan

Penjelasan:

Nomor port 4000 berasal dari variabel `PORT` di berkas `.env`, yang dibaca `dotenv` lalu dipakai pada baris `const PORT = process.env.PORT || 3000` di `app.js`. Bila port tersebut sudah dipakai aplikasi lain, Node.js akan gagal start dengan pesan `EADDRINUSE` sehingga server tidak berjalan; karena itu saya perlu mengganti nilai `PORT` atau mematikan aplikasi yang memakainya.

Bab 4 Pengujian Setiap Endpoint

Pengujian dibagi dua: lima skenario berhasil (T01 sampai T05) dan empat skenario gagal (N01 sampai N04). Skenario berhasil dilakukan **berurutan** karena T04 dan T05 memakai data yang dibuat pada T03. Setiap skenario cukup **satu** tangkapan layar yang memuat permintaan dan respons sekaligus; semua tulisan harus terbaca.

Catatan

Bunyi pesan boleh berbeda dengan contoh sesuai kode Anda, tetapi respons harus berupa JSON dan kode status harus tepat. Jika pesan Anda berbeda, tulis pesan yang sebenarnya pada kolom "Status hasil uji".

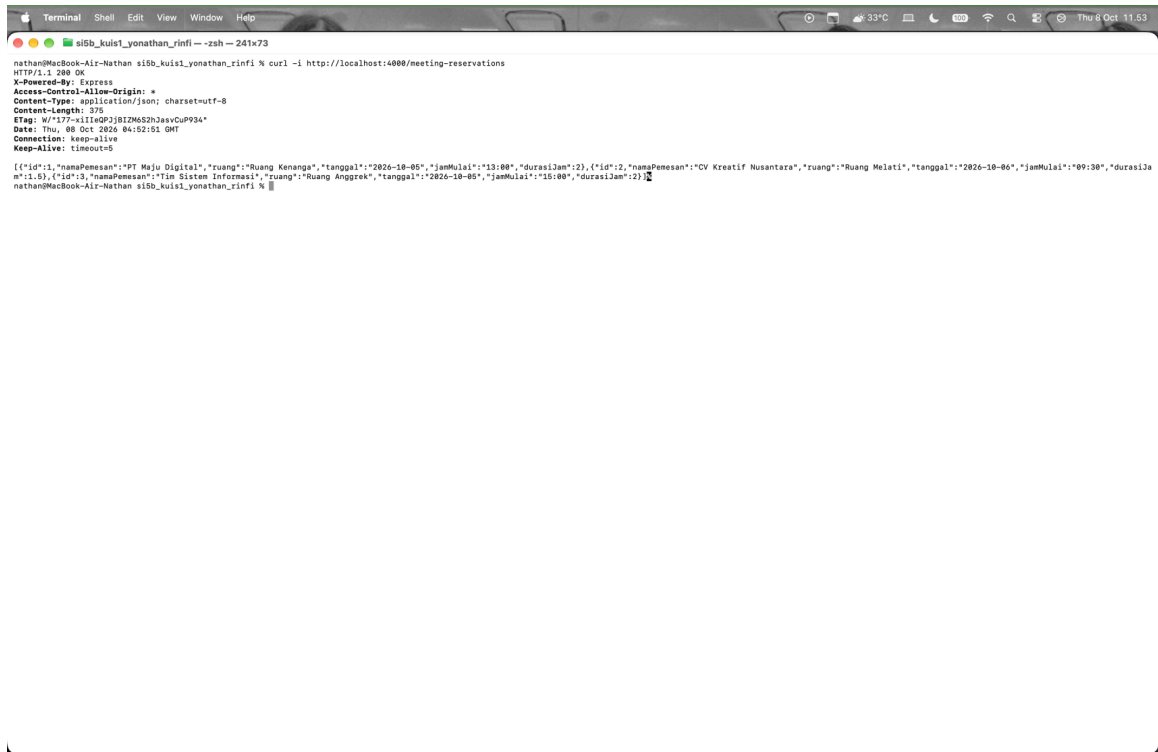
Gunakan ID 99 (atau ID lain yang pasti tidak ada) untuk skenario "tidak ditemukan".

Bagian A. Skenario berhasil

T01 Menampilkan semua data (GET /meeting-reservations)

Harus terlihat pada screenshot: metode GET, alamat lengkap, kode status, dan isi respons berupa larik JSON.

Permintaan (request)	GET /meeting-reservations
Header	(tidak ada)
Body	(tidak ada)
Hasil yang diharapkan	200 OK; isi berupa larik JSON seluruh data.
Status hasil uji	200 OK; respons berupa larik JSON berisi 3 data reservasi.
Lulus?	☒ Ya ☐ Tidak



Gambar 4. Hasil GET semua data

Gambar 4. Hasil GET semua data

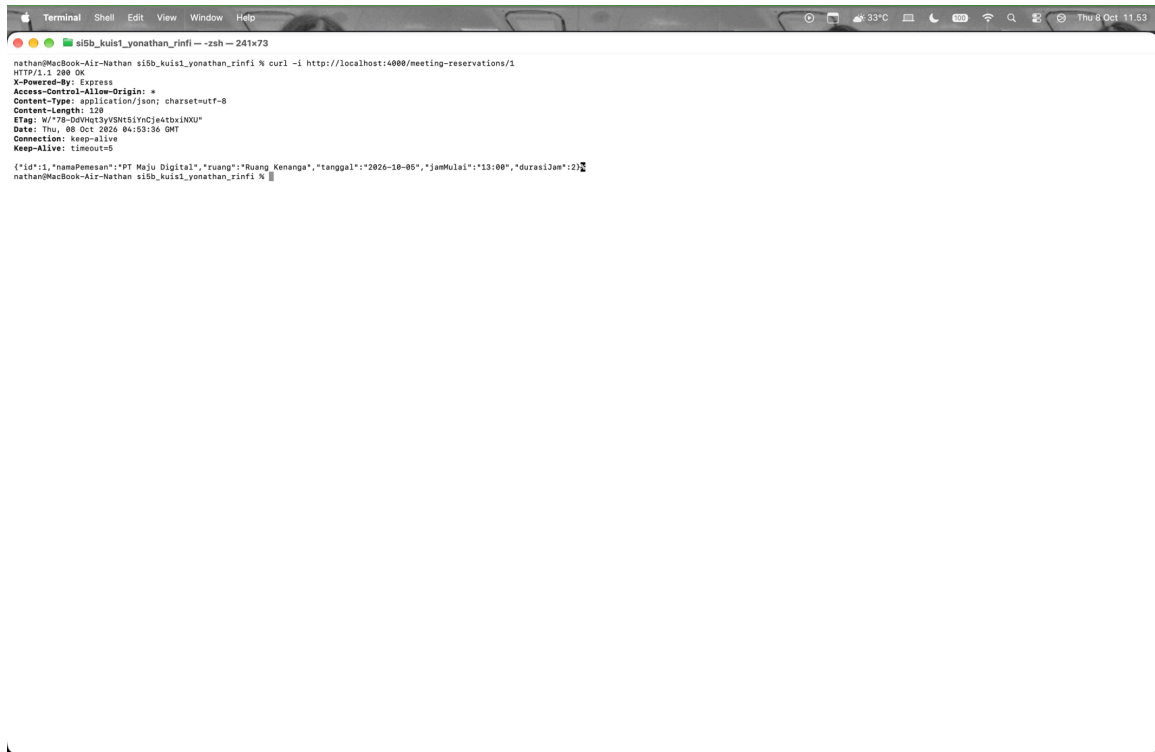
Penjelasan:

Endpoint ini membalas 200 OK karena permintaan berhasil dan seluruh data dikirim sebagai larik JSON. Urutan pemanggilannya: `route router.get("/meeting-reservations", ...)` memanggil fungsi `getAllReservations` di controller, kemudian controller memanggil `reservationModel.findAll(tanggal)` di model untuk mengambil datanya.

T02 Menampilkan satu data berdasarkan ID

Harus terlihat pada screenshot: ID pada alamat dan objek tunggal pada respons.

Permintaan (request)	GET /meeting-reservations/1
Header	(tidak ada)
Body	(tidak ada)
Hasil yang diharapkan	200 OK; satu objek JSON dengan id yang diminta.
Status hasil uji	200 OK; satu objek JSON data id 1 atas nama PT Maju Digital.
Lulus?	[X] Ya [] Tidak



```
nathan@MacBook-Air-Nathan: ~ % curl -i http://localhost:4000/meeting-reservations/1
HTTP/1.1 200 OK
X-Powered-By: Express
Access-Control-Allow-Origin: *
Content-Type: application/json; charset=utf-8
Content-Length: 120
Etag: w/"78-60vht3y9vht3jncjettbxiXKU"
Date: Thu, 08 Oct 2026 04:53:36 GMT
Connection: keep-alive
Keep-Alive: timeout=60

{"id":1,"namaPemesan":"PT Maju Digital","ruang":"Ruang Kenanga","tanggal":"2026-10-05","jamMulai":"13:00","durasiJam":2}
```

Gambar 5. Hasil GET berdasarkan ID

Gambar 5. Hasil GET berdasarkan ID

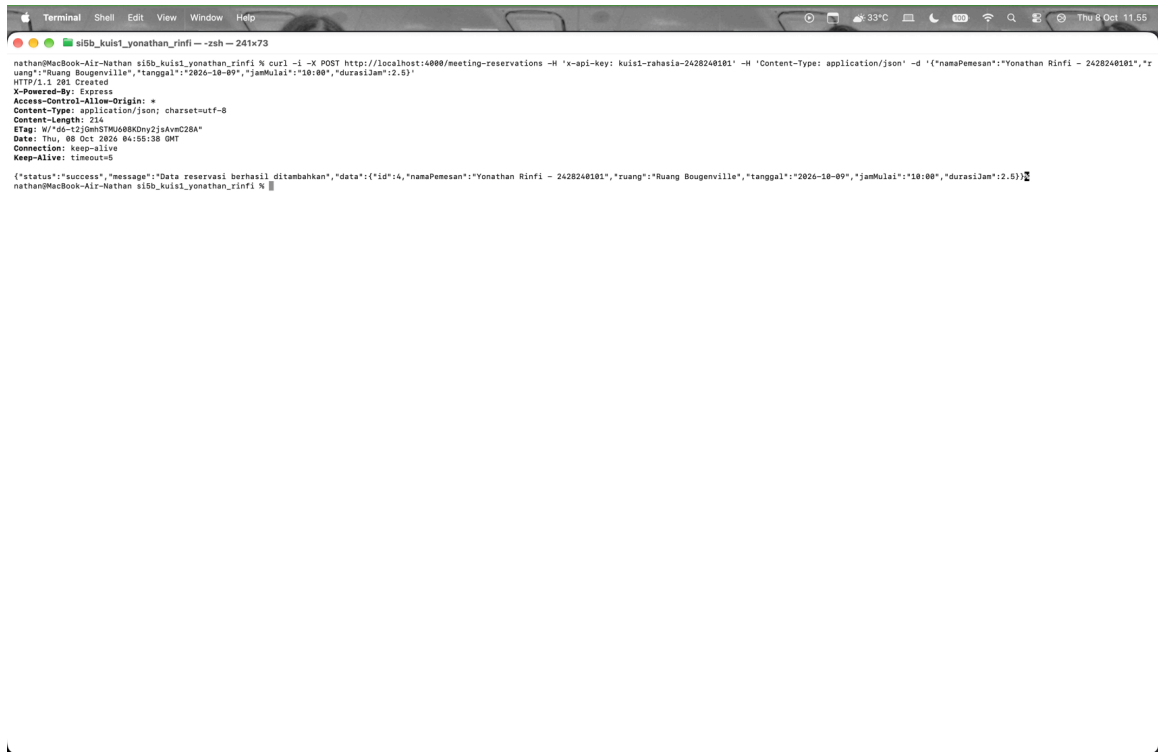
Penjelasan:

ID diambil dari alamat melalui `req.params.id` pada fungsi `getReservationById`. Nilai tersebut masih berupa teks, jadi saya ubah dengan `Number.parseInt(req.params.id, 10)` agar bisa dibandingkan dengan properti `id` yang bertipe angka di model. Tanpa diubah, perbandingan ketat (`===`) akan selalu gagal sehingga data yang ada pun dianggap tidak ditemukan.

T03 Menambah data baru (POST)

Harus terlihat pada screenshot: header x-api-key dan Content-Type, body JSON, status 201, serta objek baru lengkap dengan id. Gunakan data identitas Anda sendiri pada salah satu field teks (misalnya nama atau NIM).

Permintaan (request)	POST /meeting-reservations
Header	x-api-key: kuis1-rahasia-2428240101 Content-Type: application/json
Body	{"namaPemesan":"Yonathan Rinfi - 2428240101","ruang":"Ruang Bougenville","tanggal":"2026-10-09","jamMulai":"10:00","durasiJam":2.5}
Hasil yang diharapkan	201 Created; objek baru beserta id yang dibuat server.
Status hasil uji	201 Created; data baru dengan id 4 dan namaPemesan "Yonathan Rinfi - 2428240101".
Lulus?	☒ Ya ☐ Tidak



```
nathan@MacBook-Air-Nathan: ~/Documents/si5b_kuis1_yonathan_rinfi % curl -i -X POST http://localhost:4000/meeting-reservations -H 'x-api-key: kuisi-rahasia-2428240101' -H 'Content-Type: application/json' -d '{("namaPemesan": "Yonathan Rinfi - 2428240101", "ruang": "Ruang Bougenville", "tanggal": "2026-10-09", "jamMulai": "18:00", "durasiJam": 2.5)}'
```

```
HTTP/1.1 201 Created
X-Powered-By: Express
Access-Control-Allow-Origin: *
Content-Type: application/json; charset=utf-8
Content-Length: 214
Etag: W/"d6-12j0mSTMU608K0ny2jsAvnc28A"
Date: Thu, 08 Oct 2026 04:55:38 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{"status": "success", "message": "Data reservasi berhasil ditambahkan", "data": {"id": 4, "namaPemesan": "Yonathan Rinfi - 2428240101", "ruang": "Ruang Bougenville", "tanggal": "2026-10-09", "jamMulai": "18:00", "durasiJam": 2.5}}
```

Gambar 6. Hasil POST data baru

Gambar 6. Hasil POST data baru

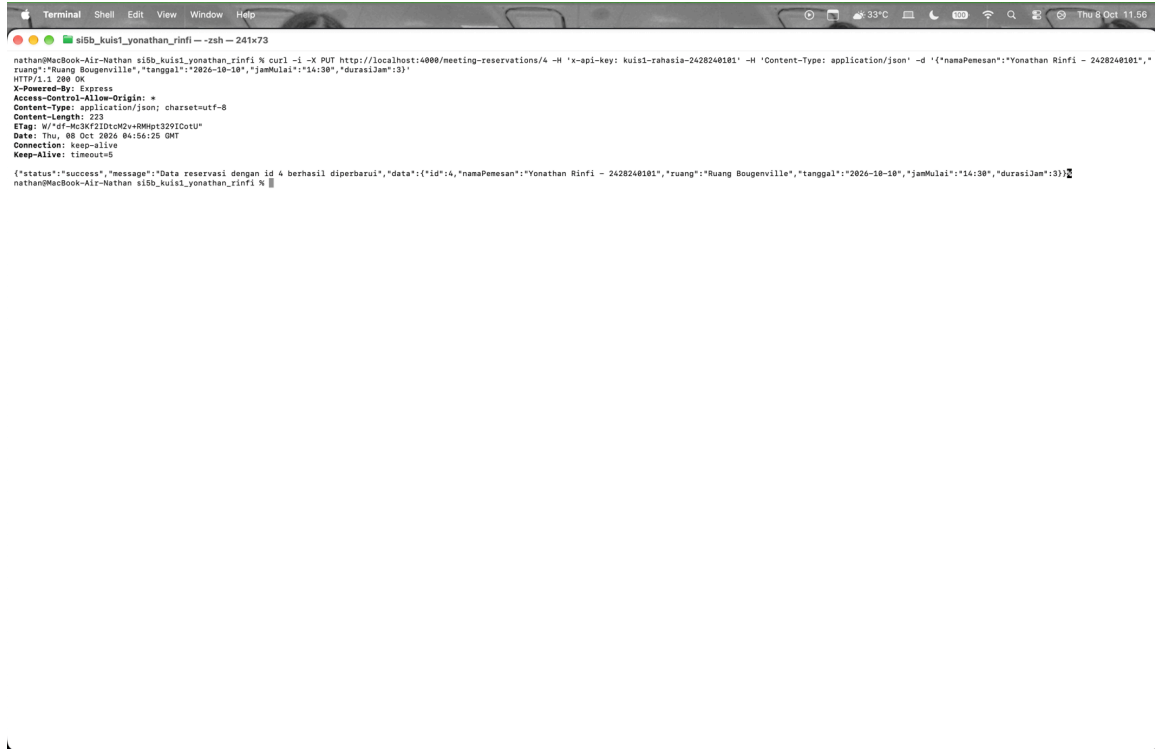
Penjelasan:

Header `x-api-key` diperiksa middleware `cekApiKey` sebelum request masuk ke controller, sehingga permintaan tanpa kunci yang benar tidak pernah sampai ke `createReservation`. `express.json()` berperan membaca body berformat JSON menjadi objek JavaScript yang dapat diakses lewat `req.body`. Status yang dipakai adalah 201 Created, bukan 200, karena permintaan ini berhasil membuat sumber daya baru di server.

T04 Mengubah data (PUT)

Harus terlihat pada screenshot: ID data baru pada alamat, header lengkap, body berisi field yang diubah, dan objek hasil perubahan.

Permintaan (<i>request</i>)	PUT /meeting-reservations/4
Header	x-api-key: kuis1-rahasia-2428240101 Content-Type: application/json
Body	{"namaPemesan":"Yonathan Rinfi - 2428240101","ruang":"Ruang Bougenville","tanggal":"2026-10-10","jamMulai":"14:30","durasiJam":3}
Hasil yang diharapkan	200 OK; objek dengan field yang sudah berubah.
Status hasil uji	200 OK; data id 4 berubah, tanggal menjadi 2026-10-10 dan durasiJam menjadi 3.
Lulus?	☒ Ya ☐ Tidak



Gambar 7. Hasil PUT mengubah data

Gambar 7. Hasil PUT mengubah data

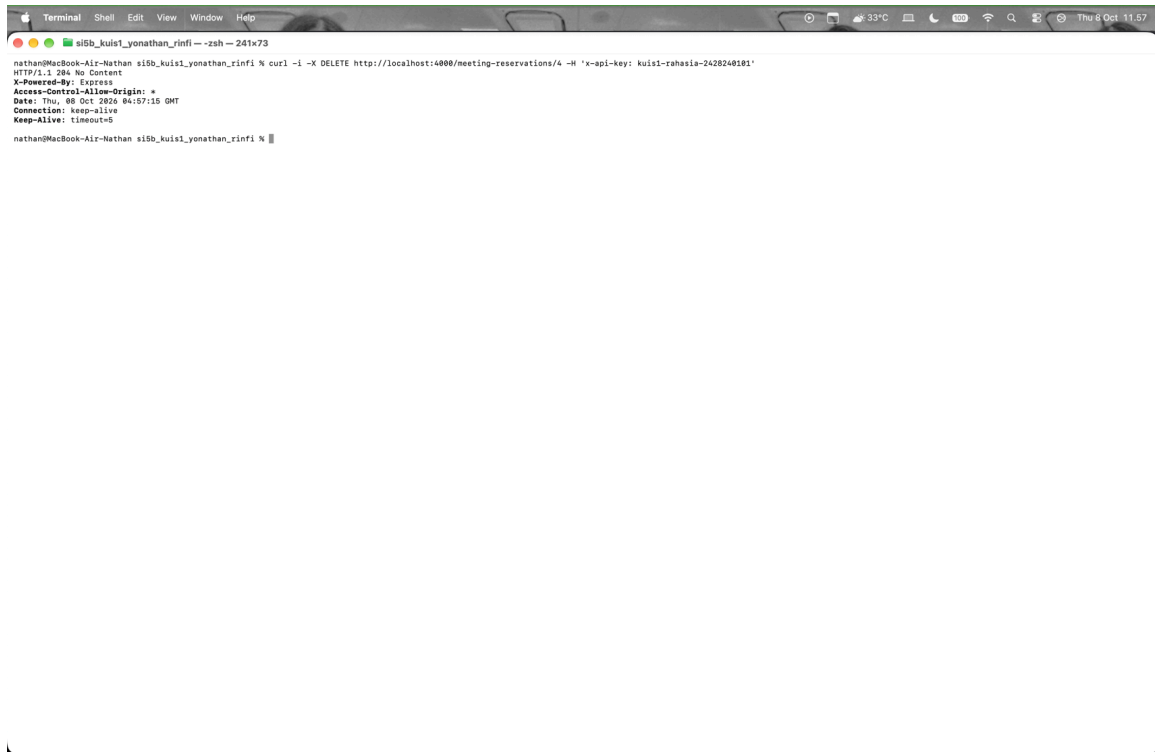
Penjelasan:

Field yang berubah adalah tanggal, dari 2026-10-09 menjadi 2026-10-10, dan durasiJam dari 2.5 menjadi 3, sedangkan id dan namaPemesan tetap. Perubahan dikerjakan fungsi `reservationModel.update(id, fields)` di layer model, yang mengganti isi array pada indeks hasil `findIndex`.

T05 Menghapus data (DELETE)

Harus terlihat pada screenshot: metode DELETE, header x-api-key, status 204, dan respons tanpa isi.

Permintaan (request)	DELETE /meeting-reservations/4
Header	x-api-key: kuis1-rahasia-2428240101
Body	(tidak ada)
Hasil yang diharapkan	204 No Content; tanpa isi respons.
Status hasil uji	204 No Content; respons tanpa isi.
Lulus?	[X] Ya [] Tidak



Gambar 8. Hasil DELETE data

Gambar 8. Hasil DELETE data

Penjelasan:

Status 204 No Content berarti permintaan berhasil tetapi server tidak mengirim isi respons apa pun. Responsnya sengaja kosong karena data yang diminta sudah terhapus sehingga tidak ada lagi yang perlu dikirim. Ini sesuai ketentuan Kuis 1, meskipun pada Tugas 1 endpoint ini semula membalas 200 beserta pesan JSON.

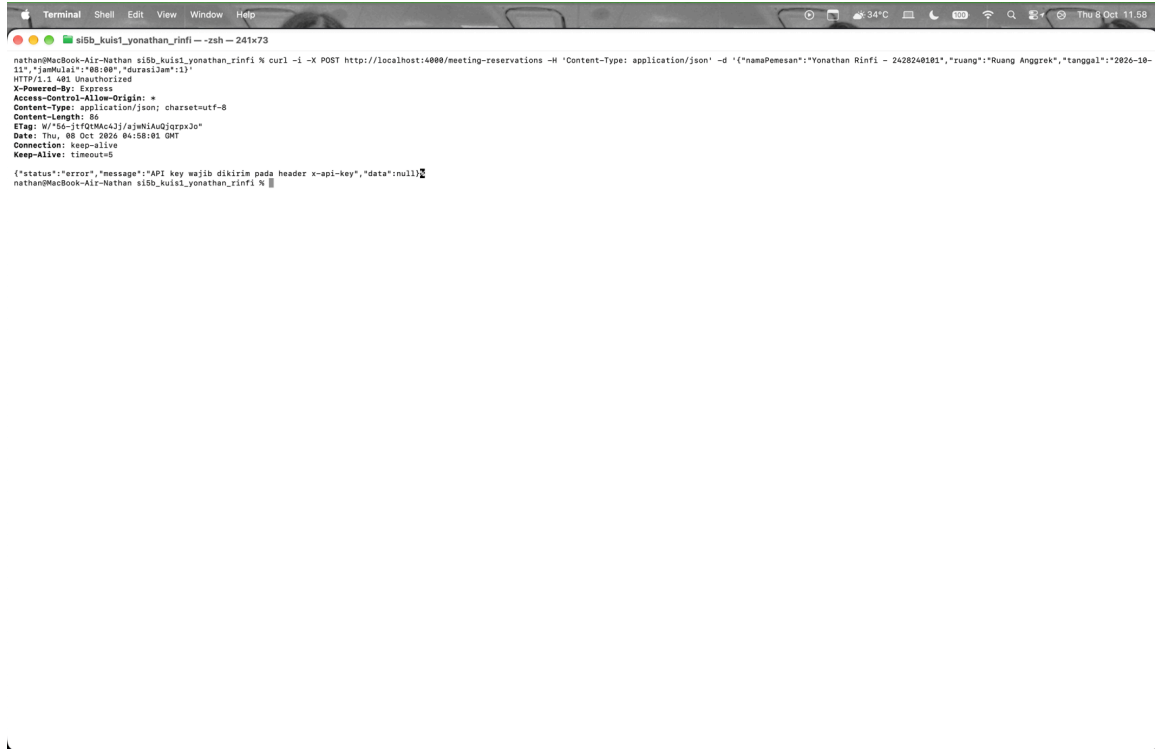
Bagian B. Skenario gagal

Skenario gagal membuktikan bahwa API menolak permintaan yang salah dengan kode status dan pesan yang tepat, bukan runtuh atau membalas halaman HTML berisi jejak error.

N01 Menambah data tanpa API key

Harus terlihat pada **screenshot**: permintaan POST tanpa header x-api-key dan respons 401.

Permintaan (<i>request</i>)	POST /meeting-reservations (tanpa x-api-key)
Header	Content-Type: application/json
Body	{"namaPemesan":"Yonathan Rinf - 2428240101","ruang":"Ruang Anggrek","tanggal":"2026-10-11","jamMulai":"08:00","durasiJam":1}
Hasil yang diharapkan	401 Unauthorized; pesan JSON; data tidak bertambah.
Status hasil uji	401 Unauthorized; pesan "API key wajib dikirim pada header x-api-key".
Lulus?	☒ Ya ☐ Tidak



Gambar 9. POST tanpa API key ditolak

Gambar 9. POST tanpa API key ditolak

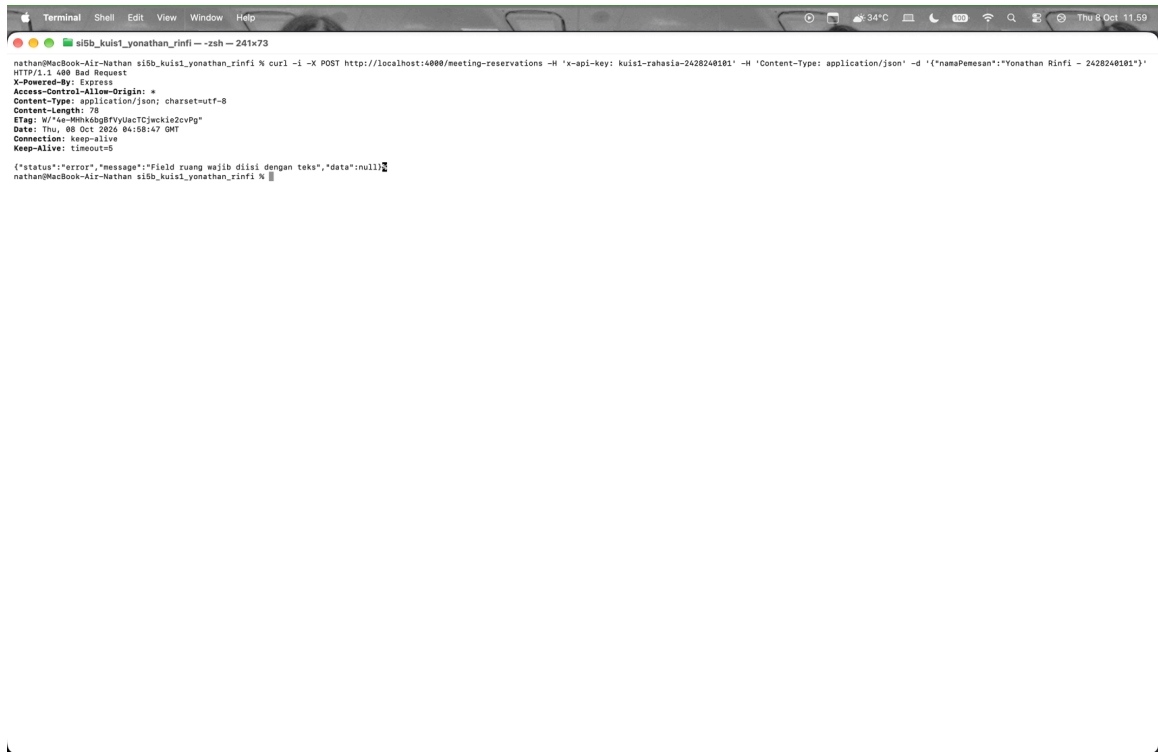
Penjelasan:

Yang menolak permintaan adalah middleware cekApiKey. Karena permintaan tidak menyertakan header x-api-key, fungsi itu langsung mengirim status 401 dan tidak memanggil next(), sehingga createReservation tidak pernah dijalankan dan data tidak bertambah.

N02 Menambah data dengan body tidak lengkap

Harus terlihat pada screenshot: body yang sengaja kekurangan field wajib dan respons 400.

Permintaan (request)	POST /meeting-reservations
Header	x-api-key: kuis1-rahasia-2428240101 Content-Type: application/json
Body	{"namaPemesan": "Yonathan Rinfir - 2428240101"}
Hasil yang diharapkan	400 Bad Request; pesan menyebut field yang wajib.
Status hasil uji	400 Bad Request; pesan "Field ruang wajib diisi dengan teks".
Lulus?	[X] Ya [] Tidak



```
nathan@MacBook-Air-Nathan si5b_kuis1_yonathan_rinfi % curl -i -X POST http://localhost:4000/meeting-reservations -H 'x-api-key: kuisi-rahasia-2428240181' -H 'Content-Type: application/json' -d '{"namaPemesan":"Yonathan Rinfi - 2428240181"}'
HTTP/1.1 400 Bad Request
X-Powered-By: Express
Access-Control-Allow-Origin: *
Content-Type: application/json; charset=utf-8
Content-Length: 78
ETag: w/"ae-8bmk2bg8f7yUacTJockla2cvPg"
Date: Thu, 08 Oct 2025 06:58:47 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{"status":"error","message":"field ruang wajib diisi dengan teks","data":null}
```

Gambar 10. POST dengan body tidak lengkap

Gambar 10. POST dengan body tidak lengkap

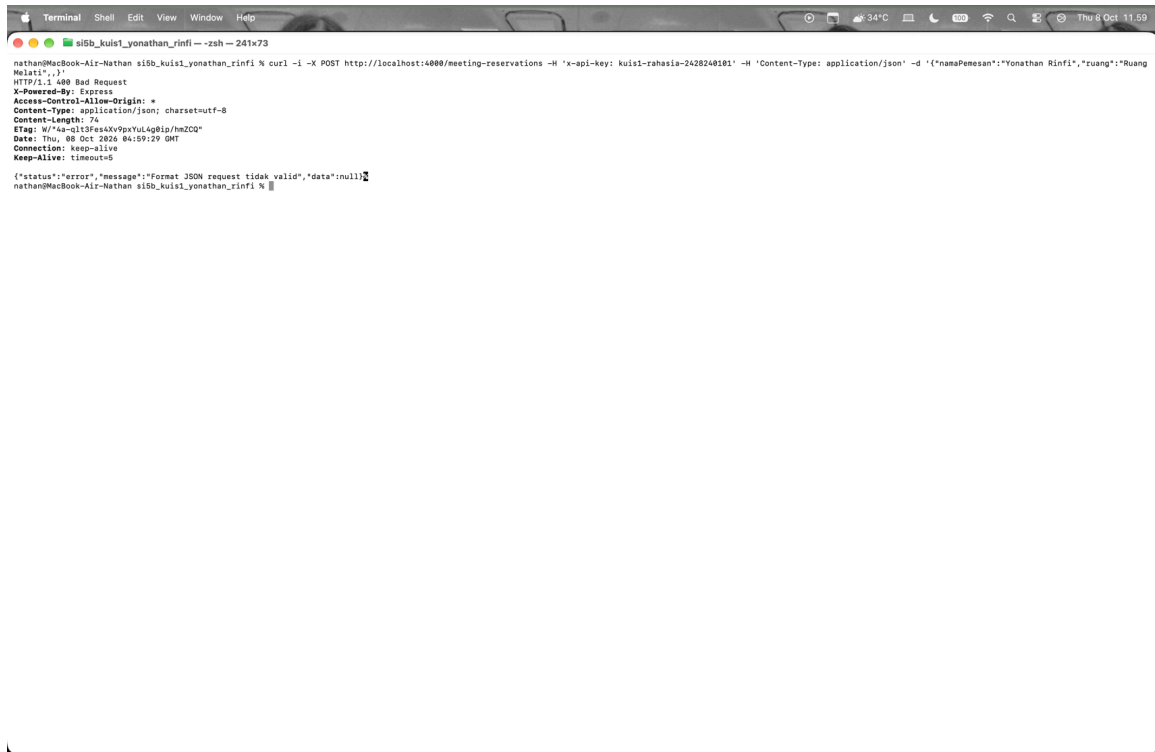
Penjelasan:

Validasi ini ditulis di layer controller, tepatnya pada fungsi `validateReservation` yang dipanggil oleh `createReservation`. Kode status 400 dipakai karena permintaan tidak valid: body hanya memuat `namaPemesan`, sedangkan field `ruang`, `tanggal`, `jamMulai`, dan `durasiJam` wajib ada.

N03 Mengirim JSON yang rusak

Harus terlihat pada screenshot: body dengan tanda baca salah (misalnya koma berlebih) dan respons 400 berformat JSON.

Permintaan (<i>request</i>)	POST /meeting-reservations
Header	x-api-key: kuis1-rahasia-2428240101 Content-Type: application/json
Body	{"namaPemesan":"Yonathan Rinfi","ruang":"Ruang Melati",,}
Hasil yang diharapkan	400 Bad Request; pesan "Format JSON tidak valid" (bukan halaman HTML).
Status hasil uji	400 Bad Request; pesan "Format JSON request tidak valid".
Lulus?	☒ Ya ☐ Tidak



Gambar 11. POST dengan JSON rusak

Gambar 11. POST dengan JSON rusak

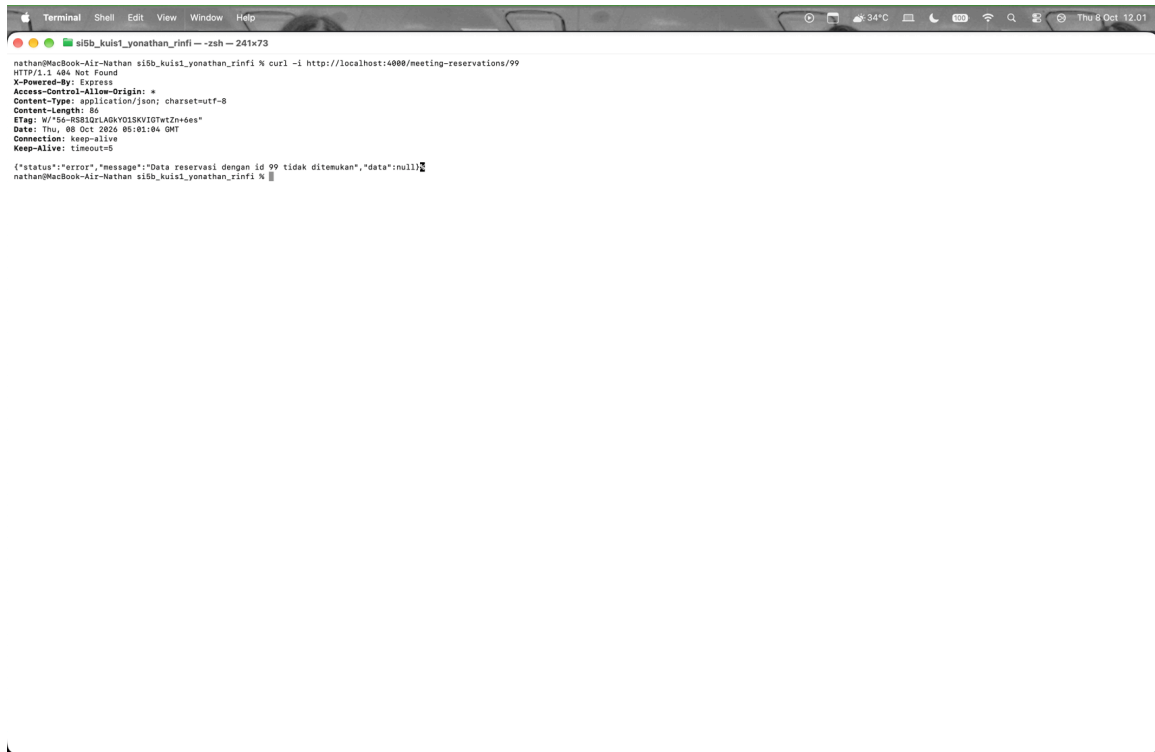
Penjelasan:

Yang mendeteksi adalah `express.json()`; body dengan koma berlebih membuatnya melempar `SyntaxError`. Error itu ditangkap middleware `errorHandler`, yang mengenali `err instanceof SyntaxError` dan `err.status === 400`, lalu mengubahnya menjadi respons JSON dengan status 400, bukan halaman HTML berisi jejak error.

N04 Mengambil ID yang tidak ada

Harus terlihat pada screenshot: ID yang jelas tidak ada (misalnya 99) dan respons 404.

Permintaan (request)	GET /meeting-reservations/99
Header	(tidak ada)
Body	(tidak ada)
Hasil yang diharapkan	404 Not Found; pesan JSON.
Status hasil uji	404 Not Found; pesan "Data reservasi dengan id 99 tidak ditemukan".
Lulus?	[X] Ya [] Tidak



Gambar 12. GET ID tidak ada

Gambar 12. GET ID tidak ada

Penjelasan:

Alurnya: `route router.get("/meeting-reservations/:id")` memanggil `getReservationById`, yang kemudian memanggil `reservationModel.findById(99)`. Karena tidak ada data dengan id tersebut, model mengembalikan null, lalu controller membalas 404 beserta pesan JSON. Jadi pesan 404 dibuat dan dikirim oleh controller.

Bab 5 Rekapitulasi Hasil Uji

Salin hasil setiap skenario ke tabel berikut. Tabel ini dibandingkan dosen dengan tangkapan layar pada Bab 4.

ID	Skenario	Status harapan	Status hasil	Lulus
T01	Menampilkan semua data	200	200 OK	Ya
T02	Menampilkan satu data berdasarkan ID	200	200 OK	Ya
T03	Menambah data baru	201	201 Created	Ya
T04	Mengubah data	200	200 OK	Ya
T05	Menghapus data	204	204 No Content	Ya
N01	Menambah data tanpa API key	401	401 Unauthorized	Ya
N02	Menambah data dengan body tidak lengkap	400	400 Bad Request	Ya
N03	Mengirim JSON yang rusak	400	400 Bad Request	Ya
N04	Mengambil ID yang tidak ada	404	404 Not Found	Ya

Bab 6 Repositori Kode

Kode dinilai dari repositori GitHub, bukan dari tangkapan layar. Pastikan repositori dapat dibuka dosen (publik atau dosen diundang).

Tautan repositori	https://github.com/Nathan-rar/si5b_kuis1_yonathan_rinfi
Pesan commit terakhir	Refactor RESTful API Reservasi Ruang Rapat ke arsitektur routes, controllers, models, dan middlewares
Folder routes, controllers, models, middlewares ada	☒ Ya ☐ Tidak
.env tidak terunggah (ada di .gitignore)	☒ Ya ☐ Tidak
.env.example ada	☒ Ya ☐ Tidak

Bab 7 Refleksi

Jawablah dengan kalimat Anda sendiri, dua sampai tiga kalimat, dan kaitkan dengan kode topik Anda.

Pertanyaan 1

Jelaskan perbedaan peran Route, Controller, dan Model pada kode Anda. Sebutkan satu contoh baris untuk masing-masing.

Penjelasan:

Route hanya bertugas memetakan alamat ke fungsi, contohnya baris `router.get("/meeting-reservations", reservationController.getAllReservations)`. Controller menangani request dan menyusun response, contohnya `res.status(200).json(reservations)` di dalam `getAllReservations`. Model menyimpan data beserta logikanya, contohnya baris `const reservation = meetingReservations.find((item) => item.id === id)` di `findById`, tanpa menyentuh `req` maupun `res`.

Pertanyaan 2

Apa keuntungan error handler terpusat dibandingkan menulis `res.status(404)` di setiap route? Tunjukkan buktinya dari pengujian.

Penjelasan:

Dengan error handler terpusat, penanganan error cukup ditulis satu kali untuk seluruh aplikasi sehingga tidak perlu menyalin `res.status(404)` di setiap route, dan bentuk responsnya tetap konsisten berupa JSON. Buktinya dari pengujian: skenario N03 yang mengirim JSON rusak tetap dibalas JSON "Format JSON request tidak valid" dengan status 400 oleh errorHandler, bukan halaman HTML berisi jejak error.

Lampiran A Daftar Periksa Sebelum Mengumpulkan

No	Butir yang diperiksa	Ya
1	Seluruh tanda [. . .] sudah diganti dengan topik Tugas 1 saya	Ya
2	Semua tangkapan layar berasal dari komputer saya dan terbaca jelas	Ya
3	Setiap gambar memiliki nomor, keterangan, dan penjelasan	Ya
4	Identitas saya (nama atau NIM) muncul pada data uji POST	Ya
5	Sembilan skenario pada Bab 4 sudah lengkap, termasuk status hasil uji	Ya
6	Tabel rekapitulasi sesuai dengan tangkapan layar	Ya
7	Tautan repositori pada sampul dapat dibuka dan .env tidak terunggal	Ya
8	Berkas disimpan sebagai PDF dengan nama sesuai ketentuan	Ya

Lampiran B Rubrik Penilaian (Usulan)

Bobot di bawah ini adalah usulan awal dan dapat diubah dosen sebelum dibagikan ke mahasiswa.

Komponen	Kriteria penilaian penuh	Bobot
Instalasi dan struktur folder (Bab 1 sampai 3)	Instalasi, struktur folder, dan server berjalan benar	15
Ringkasan kode dan kode di repositori (Bab 2 dan 6)	Tanggung jawab tiap layer tidak tercampur; penjelasan menyebut kode sendiri; .env tidak terunggal	30
Pengujian skenario berhasil (T01 sampai T05)	Status dan respons sesuai; penjelasan benar	25
Pengujian skenario gagal (N01 sampai N04)	Kode status dan pesan JSON sesuai; penjelasan alur error benar	20
Rekapitulasi dan refleksi (Bab 5 dan 7)	Rekap konsisten dengan bukti; jawaban pribadi	10
Total		100